



BITS Pilani
Pilani Campus

Software Engineering

CS 11 Design Patterns



BITS Pilani
Pilani Campus



BSDCH ZC343 - Software Engineering

Coverage



- Design Patterns
- Pattern based design process
- Architecture, Component, UI design patterns
- Web application design patterns

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Text Ref.



T1: Pressman, R.S., Software Engineering: A Practitioner's Approach

Ch 12 Pattern Based Design

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005



BITS Pilani
Pilani Campus



Pattern Based Design

Pattern Based Design



- Finding a set of proven solutions to a clearly delineated set of problems.
- Each problem and its solution is described by a design pattern that has been catalogued and vetted by other software engineers who have encountered the problem and implemented the solution while designing other applications.
- Each design pattern provides you with a proven approach to one part of the problem to be solved.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Design Pattern



A three-part rule which expresses a relation between

- a certain context,
- a problem, and
- a solution

KEY POINT

Forces are those characteristics of the problem and attributes of the solution that constrain the way in which the design can be developed.



quote:

"Our responsibility is to do what we can, learn what we can, improve the solutions, and pass them on."

Richard P. Feynman

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Design Pattern



- Solves a problem
 - capture solutions, not just abstract principles or strategies.
- It is a proven concept.
- The solution isn't obvious.
 - The best patterns generate a solution to a problem indirectly not just from first principles.
- It describes a relationship
 - Patterns don't just describe modules but describe deeper system structures and mechanisms.
- The pattern has a significant human component (minimize human intervention).
 - Appeal to aesthetics and utility.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Kinds of Patterns



- Nongenerative pattern
- Architectural patterns
- Data patterns
- Component patterns
- Interface design patterns
- WebApp patterns
- Idioms

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Nongenerative pattern



- Describes a context and a problem but it does not provide any clear-cut solution.
- Provides reusable design solutions for common software design problems without generating code automatically.
- Improves software quality by enhancing maintainability, flexibility, and code reusability.
- Defines best practices for organizing classes, objects, and their interactions.
- Examples: Singleton, Observer, Factory, MVC

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Architectural patterns



- Broad-based design problems that are solved using a structural approach.
- Provide the overall structure of a software system by defining how major components are organized and interact.
- Improve scalability, maintainability, and reusability by using proven architectural solutions.
- Examples: Layered Architecture, Client–Server, Microservices, Model–View–Controller (MVC), Event-Driven Architecture.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005



- Describe recurring data-oriented problems and the data modelling solutions that can be used to solve them.
- Define how data is organized, stored, accessed, and managed within a software system.
- Improve data consistency, scalability, and performance by using standardized data organization techniques.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Component patterns



- Define how software is divided into reusable, independent components with specific responsibilities.
- Promote modularity, reusability, and easy maintenance by allowing components to be developed and updated independently.
- Improve scalability and flexibility through well-defined interfaces between components.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Interface design patterns



- Describe common user interface problems and their solution with a system of forces that includes the specific characteristics of end users.
- Define how users interact with a software application through consistent and intuitive interfaces.
- Improve usability, accessibility, and user experience (UX) by following standard interface design practices.
- Ensure consistency across screens, menus, navigation, and user interactions.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

WebApp patterns



- Address a problem set that is encountered when building WebApps and often incorporates many of the other pattern's categories just mentioned.
- Provide reusable design solutions for developing scalable, secure, and maintainable web applications.
- Define how web pages, business logic, and databases interact within a web application.
- Improve performance, user experience, and code reusability by following proven web development practices.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Idioms



- Describe how to implement all or part of a specific algorithm or data structure for a software component within the context of a specific programming language.
- Idioms are low-level programming patterns that provide language-specific solutions to common coding problems.
- They improve code readability, reusability, and efficiency by following best programming practices.
- Idioms are specific to a programming language (e.g., Java, C++, Python) rather than the overall software architecture.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Framework



1. Design patterns are more abstract than frameworks.
2. Design patterns are smaller architectural elements than frameworks.
3. Design patterns are less specialized than frameworks.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

A Pattern Organisation Table



	Database	Application	Implementation	Infrastructure
Data/Content				
Problem statement ...	PatternName(s)		PatternName(s)	
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...	PatternName(s)			PatternName(s)
Architecture				
Problem statement ...		PatternName(s)		
Problem statement ...		PatternName(s)		PatternName(s)
Problem statement ...				
Component-level				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...				PatternName(s)
Problem statement ...		PatternName(s)	PatternName(s)	
User interface				
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	
Problem statement ...		PatternName(s)	PatternName(s)	

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Object-oriented design patterns – Gamma et. al. Gang of Four (GoF)



- Creational patterns
- Structural patterns
- Behavioural patterns

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Creational patterns



“creation, composition, and representation” of objects.
encapsulate knowledge about which concrete classes the system uses

hide how instances of these classes are created and put together.

Creational patterns provide mechanisms that make the instantiation of objects easier within a system and enforce “constraints on the type and number of objects that can be created within a system”

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Creational Patterns



- Abstract factory pattern: centralize decision of what factory to instantiate.
- Factory method pattern: centralize creation of an object of a specific type choosing one of several implementations.
- Builder pattern: separate the construction of a complex object from its representation so that the same construction process can create different representations.
- Prototype pattern: used when the inherent cost of creating a new object in the standard way (e.g., using the “new” keyword) is prohibitively expensive for a given application.
- Singleton pattern: restrict instantiation of a class to one object.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Structural patterns



- How classes and objects are organized and integrated to build a larger structure.
- Establish relationships between entities within a system.
- Objects suggest techniques for combining objects within other objects or integrating objects into a larger structure.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Structural patterns



- Adapter pattern: “adapts” one interface for a class into one that a client expects.
- Aggregate pattern: a version of the composite pattern with methods for aggregation of children.
- Bridge pattern: decouple an abstraction from its implementation so that the two can vary independently.
- Composite pattern: a tree structure of objects where every object has the same interface.
- Container pattern: create objects for the sole purpose of holding other objects and managing them.
- Proxy pattern: a class functioning as an interface to another thing.
- Pipes and filters: a chain of processes where the output of each process is the input of the next.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Behavioural patterns



- Address problems associated with the assignment of responsibility between objects and the way communication is affected between objects.
- Defines communication and interaction among objects.
- Promotes loose coupling and flexible behavior in software systems.
- Examples: Observer, Strategy, Command, State, Chain of Responsibility.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Behavioural patterns /cont...



- Chain of responsibility pattern: Command objects are handled or passed on to other objects by logic-containing processing objects.
- Command pattern: Command objects encapsulate an action and its parameters.
- Event listener: Data are distributed to objects that are registered to receive them.
- Interpreter pattern: Implement a specialized computer language to rapidly solve a specific set of problems.
- Iterator pattern: Iterators are used to access the elements of an aggregate object sequentially without exposing its underlying representation.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Behavioural patterns ...cont



- Mediator pattern: Provides a unified interface to a set of interfaces in a subsystem.
- Visitor pattern: A way to separate an algorithm from an object.
- Single-serving visitor pattern: Optimize the implementation of a visitor that is allocated, used only once, and then deleted.
- Hierarchical visitor pattern: Provide a way to visit every node in a hierarchical data structure such as a tree.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Framework



1. Design patterns are more abstract than frameworks.

Frameworks can be embodied in code, but only examples of patterns can be embodied in code. A strength of frameworks is that they can be written down in programming languages and not only studied but executed and reused directly. . . .

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Framework



2. Design patterns are smaller architectural elements than frameworks.

A typical framework contains several design patterns, but the reverse is never true.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005



3. Design patterns are less specialized than frameworks.

Frameworks always have a particular application domain. In contrast, design patterns can be used in nearly any kind of application. While more specialized design patterns are certainly possible, even these wouldn't dictate an application architecture.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Design Pattern Template



Pattern name

Problem - pattern addresses

Motivation - example of the problem

Context - environment/application domain

Forces - limitations and constraints that must be considered

Solution -solution proposed

Intent - what it does

Collaborations - how other patterns contribute to the solution

Consequences- trade-offs and consequences of using the pattern

Implementation - issues that should be considered

Known uses - examples of actual uses in real applications

Related patterns—cross-references related design patterns

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Design Tasks



1. Examine the requirements model and develop a problem hierarchy.
2. Determine if a reliable pattern language has been developed for the problem domain.
3. Beginning with a broad problem, determine whether one or more architectural patterns is available for it.
4. Using the collaborations provided for the architectural pattern, examine subsystem or component-level problems and search for appropriate patterns to address them.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

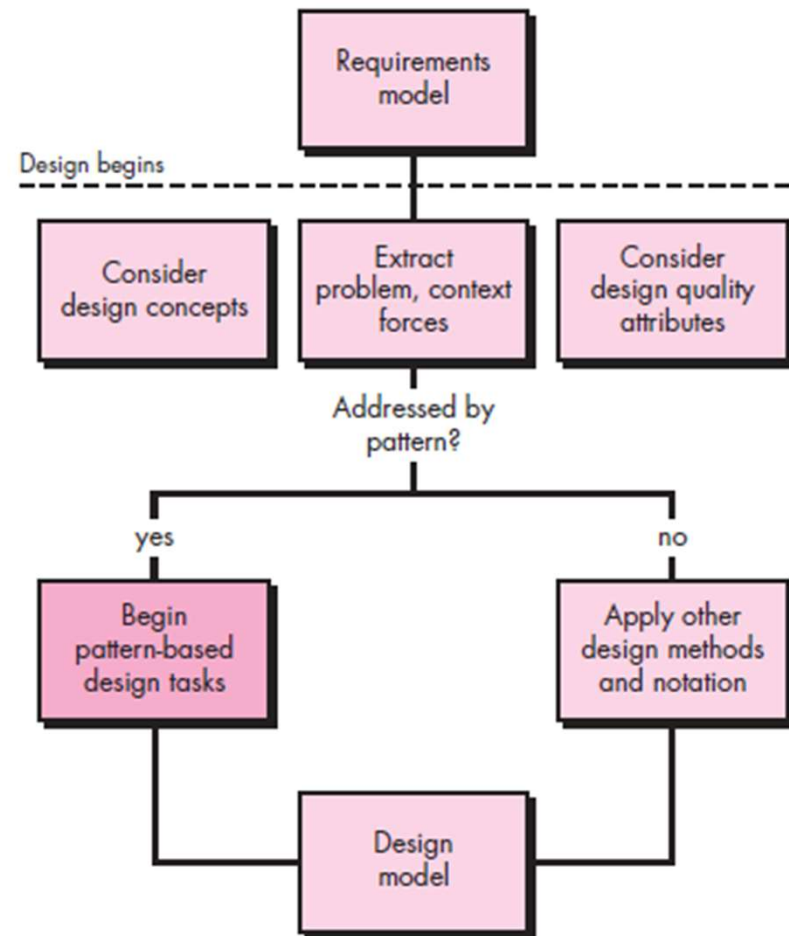
Design Tasks



5. Repeat steps 2 through 5 until all broad problems have been addressed.
6. If user interface design problems have been isolated (this is almost always the case), search the many user interface design pattern repositories for appropriate patterns.
7. Regardless of its level of abstraction, if a pattern language and/or patterns repository or individual pattern shows promise, compare the problem to be solved against the existing pattern(s) presented.
8. Be certain to refine the design as it is derived from patterns using design quality criteria as a guide.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

Pattern Based Design in Context



These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005

End



Thank you.

These courseware materials are to be used in conjunction with *Software Engineering: A Practitioner's Approach*, 6/e and are provided with permission by R.S. Pressman & Associates, Inc., copyright © 1996, 2001, 2005